



INSTITUTO TECNOLÓGICO DE SAN LUIS POTOSÍ

Ing. Sistemas Computacionales

Materia:

Ingeniería de software 8:00 - 9:00 AM

“Tarea 4.1 Pruebas del Sistema”

Integrantes:

Coronado Hernández David

Gonzales Castillo Gael Essau

Moreno Flores Braulio Edgardo

Reyna Hernández Yaotl Isaías

Maestro: Oscar Abundio Juárez Romero

Fecha de entrega: 29 de abril del 2026

28/04/2026

1. Introducción y Objetivos del Plan de Pruebas

El presente Plan de Pruebas de Software ha sido elaborado con el propósito de establecer las directrices, técnicas y metodologías para asegurar la máxima calidad en la aplicación desarrollada (PremiumBus). Esta aplicación está diseñada para gestionar la geolocalización de autobuses, venta de boletos interactiva al usuario final, validación estricta de roles (Administrador vs. Usuario Estándar) y la administración de perfiles (actualización de fotografías y acceso a un directorio general de usuarios en tiempo real).

El objetivo principal del plan es detectar, aislar, reportar, mitigar y corregir defectos en el código antes de su despliegue en el entorno de producción (Android/iOS/Web). A través de la aplicación metódica de los postulados expuestos en la obra de ingeniería referente: "Ingeniería del Software. Un Enfoque Práctico" de Roger S. Pressman (Capítulos 17, 18, 19 y 20), este documento argumenta sólidamente el marco teórico y práctico de las pruebas de unidad, de integración, de validación y sistémicas. Se garantiza así que todos los requisitos y reglas de negocio, como la inmutabilidad arquitectónica o la persistencia de sesión por Firebase, se cumplan a cabalidad.

2. Alcance del Proyecto y Marco Metodológico

El alcance de estas pruebas cubre el cien por ciento (100%) de los módulos críticos recién implementados o refactorizados en el sistema. Esto incluye la persistencia del control de inicio de sesión gestionado con Firebase Authentication, las consultas optimizadas a la base de datos no relacional de Firestore, la lógica matemática y de validación en la compra de boletos (asegurando el cumplimiento de la tarifa fija de 13.50 MXN), la validación de visibilidad asimétrica de componentes según el rol del usuario (por ejemplo, ocultando el botón "comprar boleto" si la sesión corresponde a un perfil de administrador), y las actualizaciones estéticas de la interfaz de usuario, priorizando las notificaciones interactivas Toast de formato compacto.

La metodología aplicada se basa en un paradigma de "Diseño y Testeo Atómico", donde cada componente visual o lógico se aísla, se estresa, y se le realizan aserciones (pruebas de unidad) antes de ser acoplado al árbol de dependencias principal del sistema (pruebas de integración). Este modelo iterativo e incremental mitiga el riesgo del "efecto cascada" donde un error mínimo en un módulo de bajo nivel rompe toda la aplicación.

3. Estrategias de Prueba de Software (Referencia: Capítulo 17)

3.1. Enfoque Estratégico General

En el capítulo 17 de su texto, Pressman aborda las pruebas de software desde un nivel estratégico global, estableciendo que deben construirse sobre el concepto de V&V (Verificación y Validación). La verificación responde a "¿Estamos construyendo el producto correctamente?", mientras que la validación responde a "¿Estamos construyendo el producto correcto?". Para PremiumBus, este enfoque en espiral implica iniciar desde el epicentro del sistema (bases de datos y controladores) hacia el exterior (interfaces gráficas y experiencia de usuario).

La recolección de métricas estará a cargo de un equipo ITG (Grupo Independiente de Prueba), quien, simulando el comportamiento del usuario sin estar "sesgado" por conocer el código de antemano, confirmará que las estrategias planeadas convergen en un producto confiable.

3.2. Pruebas de Unidad

Siguiendo la literatura, la prueba de unidad centra el esfuerzo de verificación en la menor porción de diseño del software: el módulo. Durante esta etapa probaremos el comportamiento algorítmico interno. Para nuestra aplicación se focalizarán en:

- **Lógica de Facturación y Compra:** Funciones aisladas que calculan la deducción matemática de 13.50 MXN del saldo total del usuario. Se enviarán vectores de prueba con saldos insuficientes, exactos y excedentes para garantizar el cumplimiento de "Early Return" propuesto por Clean Code.
- **Autenticación de Estado:** Módulos responsables de descifrar y almacenar el JWT de Firebase. Se comprobará de manera aislada que un reinicio abrupto de la aplicación no invalida la sesión si el token no ha expirado.
- **Optimización de Consultas:** Módulos de filtrado de Firebase que evitan usar índices compuestos innecesarios cuando se solicitan grandes volúmenes de usuarios para la vista de Directorio Administrativo.

3.3. Pruebas de Integración (Top-Down y Bottom-Up)

Las pruebas unitarias independientes son insuficientes si los módulos no se comunican correctamente. Pressman sugiere dos enfoques. Emplearemos una estrategia de Integración "Top-Down" para los módulos de la interfaz. Esto significa que la arquitectura de navegación de pantallas (React Navigation / Expo Router) se prueba primero empleando "Stubs" temporales en lugar de llamadas reales a la API.

Paralelamente, para la persistencia de datos aplicaremos un enfoque "Bottom-Up". Probaremos la lectura/escritura en Firebase (la base), la conectaremos a los repositorios de datos y luego a los casos de uso, minimizando fallos asíncronos imprevistos y bloqueos (race conditions).

3.4. Pruebas de Validación y del Sistema

En esta etapa validamos todo el software bajo condiciones de "caja negra" reales. Destacan las siguientes pruebas de nivel Sistema:

- **Pruebas de Recuperación (Recovery Testing):** Interrupción forzada del Wifi o Datos Móviles durante la ventana de confirmación del ticket de 13.50 MXN. Se verificará que el rollback se ejecuta, el sistema reporta la desconexión vía Toast y no se efectúan deducciones fantasmas.
- **Prueba de Rendimiento (Performance Testing):** Carga concurrente del directorio con datos simulados masivos. Evaluación del tiempo de renderizado de la UI de React Native y que no se presente "Frame dropping" notable en el scroll.

4. Prueba de Aplicaciones Convencionales (Referencia: Capítulo 18)

4.1. Fundamentos y Facilidad de Prueba (Testability)

La "Testability" o facilidad para someter un sistema a pruebas se garantiza en nuestro desarrollo acatando los principios de Separación Estricta de Responsabilidades (SoC). La capa visual es deliberadamente "tonta", lo que facilita enormemente correr pruebas sobre los controladores subyacentes sin depender de un simulador visual corriendo en segundo plano.

4.2. Pruebas de Caja Blanca

La técnica de Caja Blanca (también conocida como prueba de cristal) se realiza comprendiendo y vislumbrando el código fuente. Pressman recomienda el uso de grafos de flujo para definir el camino básico.

Análisis de Complejidad Ciclomática - Módulo "Procesar Compra": Se evaluarán las aristas y nodos condicionales. El proceso evalúa si el usuario está autenticado, si el rol permite comprar (siendo Admin = false, interrupción de ruta), si el usuario tiene un saldo ≥ 13.50 MXN y si la base de datos acepta la transacción. El diseño de pruebas garantiza que se ejecuten casos para cubrir absolutamente todos los caminos independientes del código, reduciendo así la probabilidad de condiciones "fantasma" que produzcan bugs latentes.

4.3. Pruebas de Caja Negra

Las pruebas de Caja Negra operan sobre la interfaz pública del software sin saber cómo trabaja internamente, priorizando la detección de errores funcionales o de entrada/salida. Las técnicas implementadas serán Partición de Equivalencia y Análisis de Valores Límite.

- **Valores Límite en Formularios Financieros:** Siendo el boleto 13.50, probaremos saldos del usuario en 13.49 (Debería rechazar), 13.50 (Límite exacto, debería aceptar y dejar balance en 0) y 13.51 (Acepta, balance 0.01).
- **Particiones de Equivalencia en Fotografía de Perfil:** Agruparemos las entradas válidas (Imágenes PNG o JPG menores a 2MB) y entradas inválidas (Archivos PDF, TXT, imágenes mayores a 2MB) para verificar que el sistema de validación previene fallos antes de contactar con el almacenamiento de Firebase Storage.

5. Prueba de Aplicaciones Orientadas a Objetos (Referencia: Capítulo 19)

5.1. Impacto de la Orientación a Objetos en el Testeo

Las arquitecturas tradicionales prueban funciones modulares. Sin embargo, Pressman describe cómo en sistemas Orientados a Objetos (OO), la unidad mínima testeable encapsulada es la Clase (Modelos de entidad). Conceptos como encapsulamiento, inmutabilidad de datos y polimorfismo dictan la manera de construir los casos.

5.2. Pruebas a Nivel de Clase (Modelos de Dominio)

El comportamiento de los modelos `Usuario`, `Ticket` y `GeoMarker` se pondrá a prueba en sus propios entornos. Por ejemplo, el objeto Ticket experimenta un ciclo de vida, cambiando su estado interno (atributo) de "Emitido" a "Consumido". Las pruebas asegurarán que el encapsulamiento impida que agentes externos modifiquen este estado de manera ad-hoc. Solo los métodos legítimos del objeto (por ejemplo `ticket.scan()`) podrán alterar su atributo, validando la regla de "Inmutabilidad por Defecto" de las buenas prácticas.

5.3. Pruebas de Integración O-O

La interacción mediante paso de mensajes entre objetos será escrutada a fondo a través de Pruebas Basadas en Hilos (Thread-based testing). Se ejecutará un hilo asíncrono donde un objeto `GPSController` pasa continuamente latitud y longitud a un modelo `SessionLocation`. Se medirá la tolerancia de estas instancias a retrasos en red, evaluando si

un hilo entra en un ciclo mortal infinito esperando el evento "promise resolve" o si los timeouts se administran con gracia.

6. Prueba de Aplicaciones Web y Móviles (Referencia: Capítulo 20)

6.1. Dimensiones de Calidad para Ecosistemas App/Web

Según el capítulo 20 de la bibliografía base, las WebApps y sistemas móviles enfrentan el reto constante de operar en plataformas altamente fragmentadas (diferentes navegadores, versiones de Android/iOS y tamaños de pantalla). La calidad se evalúa en confiabilidad de la red y estética de la interfaz ("Vibe Atómico").

6.2. Prueba de la Interfaz de Usuario (UI) y Usabilidad

En este sistema se ha invertido significativamente en diseño moderno. Por lo tanto, las pruebas estéticas son estrictas:

Validación de Tokens Visuales: Ninguna parte de la aplicación debe fallar en seguir las pautas de estilo. Las notificaciones interactivas tipo "Toast" fueron compactadas expresamente para no estorbar el espacio móvil. Se validará que su ancho no supere el 85% de la pantalla en dispositivos pequeños (como iPhone SE o pantallas pequeñas de Android), preservando la "Resiliencia Visual" y el "Responsive Design".

6.3. Prueba de Seguridad, Compatibilidad y Geolocalización

Configuración Multi-Plataforma: Se simularán escenarios bajo emuladores nativos de iOS y Android para asegurar que el uso de librerías como MapView cargue correctamente los mapas de Apple en iOS y Google Maps en Android, ajustando las coordenadas geográficas de forma transparente.

Pruebas de Seguridad en Auth y Bases de Datos: Se comprobarán las reglas de seguridad publicadas en el backend (Firebase Rules). Intentaremos, bajo el contexto O-O, falsificar el objeto `AuthToken` del usuario e inyectar un cambio de tarifa para evadir el cobro de 13.50. El servidor deberá identificar la discrepancia del JWT y rechazar la acción permanentemente. De esta forma, protegemos los accesos exclusivos al Directorio de Usuarios para Administradores.

7. Matrices Formales y Casos de Prueba Extendidos (Caja Negra y Blanca)

Las siguientes tablas definen el protocolo estricto que deberá seguir el departamento de aseguramiento de calidad (QA). Se han seleccionado escenarios vitales del ecosistema PremiumBus para aplicar la teoría de Ingeniería del Software vista en los capítulos previos.

7.1. Módulo de Autenticación, Acceso y Perfil de Usuario

ID	Descripción del Escenario	Datos de Precondición	Procedimiento / Pasos	Resultado Esperado (Validación)
CP-AUTH-01	Inicio de sesión feliz y retención de token	Usuario creado y red estable.	1. Ingresar credenciales correctas. 2. Acceder al sistema. 3. Cerrar la aplicación desde el administrador de tareas de OS. 4. Reabrir.	El sistema inicia sin pedir credenciales, reanudando la sesión mediante token persistente guardado localmente de forma segura.
CP-AUTH-02	Prevención de Inyecciones/Ataque.	Ninguna.	1. Ingresar como email scripts como <code><script></code>. 2. Pulsar botón login.	Bloqueo inmediato local sin petición de red, indicando "Formato de email inválido" (Clean Code Early Return).
CP-PERF-01	Carga masiva de fotografía de perfil	Usuario autenticado y API Storage operativa.	1. Clic en modificar perfil. 2. Cargar una imagen en formato .JPG válida. 3. Enviar formulario.	El archivo sube asíncronamente mostrando barra de progreso o un spinner de carga, finalmente actualiza la miniatura en tiempo real.

7.2. Módulo de Compras (\$13.50) y Roles de Acceso (Admin)

ID	Descripción del Escenario	Datos de Precondición	Procedimiento / Pasos	Resultado Esperado (Validación)
----	---------------------------	-----------------------	-----------------------	---------------------------------

CP-COMPRA-01	Comprobación límite exacto de compra de ticket.	Saldo en billetera = \$13.50 MXN (Usuario normal).	1. Ir a sección de tickets. 2. Pulsar botón "Adquirir".	La transacción es aprobada, el código QR se emite. El saldo restante reflejado del cliente es de \$0.00 MXN exactamente.
CP-COMPRA-02	Fallo previsto por insuficiencia de saldo.	Saldo en billetera = \$13.49 MXN (Usuario normal).	1. Pulsar botón "Adquirir".	Fallo procesado. Alerta Toast (Estilo Error compacto rojo) indica "Fondo insuficiente", transacción abortada en milisegundos.
CP-ROL-01	Restricción de vistas por perfil Administrador.	Cuenta tipo "Admin" logueada.	1. Inspeccionar pantalla de inicio y menú lateral.	La opción de "Comprar Boletos" ha desaparecido de la interfaz (Renderizado Condicional), cumpliendo los requerimientos de la actualización V2.
CP-ROL-02	Acceso a la información general por directorio.	Cuenta tipo "Admin" logueada.	1. Navegar a "Directorio de Usuarios".	Se despliega una lista completa optimizada sin arrojar advertencias de "composite indices" requeridos en la consola de Firebase.

7.3. Módulo de Geolocalización Activa y Reactiva

ID	Descripción del Escenario	Datos de Precondición	Procedimiento / Pasos	Resultado Esperado (Validación)
----	---------------------------	-----------------------	-----------------------	---------------------------------

CP- GEO- 01	Seguimiento ininterrumpido en tiempo real.	Dispositivo con GPS activado y permisos dados en Settings.	1. Ingresar a la sección de Mapa (Ubicación de Buses). 2. Desplazar físicamente el teléfono 10 metros.	El marcador gráfico del usuario se desliza de forma fluida reflejando el movimiento sin "saltos" exagerados (poller optimizado).
CP- GEO- 02	Corte abrupto de servicios de ubicación.	El mapa está funcionando normal.	1. Cambiar a los ajustes del sistema y desactivar el hardware GPS (Modo Avión o Apagar Ubicación). 2. Regresar a la app.	La app detecta el cambio de estado, congela la actualización y notifica cortésmente vía Toast compacto: "Se perdió señal GPS". Evita un crash fatal.

8. Entornos, Recursos y Criterios de Aceptación

Para la correcta puesta en marcha de estas pruebas se dispondrá de dispositivos virtuales (Emuladores como Android Studio o Simuladores de XCode) y dispositivos físicos reales (Smartphones de prueba dedicados). El entorno de backend se mantendrá en un servidor de Staging (Pruebas), separando la base de datos de producción para evitar contaminación de datos de clientes reales.

Métricas y Criterios de Aceptación/Cierre: De acuerdo con la etapa final descrita por Pressman, estas pruebas no pueden prolongarse indefinidamente. Se considerará el producto apto para su lanzamiento (Deploy) cuando:

- 1. La tasa de éxito en las Pruebas Críticas (Casos: CP-AUTH, CP-COMPRA, CP-ROL) alcance estrictamente el 100%.
- 2. No existan incidentes clasificados como "Severidad 1" (Caídas completas de la aplicación, fugas de datos y corrupción de saldos).
- 3. Todo el código subido al repositorio principal pase las revisiones automáticas (Linter, revisiones de arquitectura SOLID e Inmutabilidad).

Responsable / director de Proyecto
Aprobación de Ingeniería de Software

Validación de Calidad (QA)
Autorización de Fase de Pruebas